



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №7
«Шаблони «Mediator», «Facade», «Bridge»,
«Template method»
Варіант - 4

Виконала:
Студентка групи ІА-22
Третякова Ельвіра

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми.....	7
Реалізація шаблону за обраною темою.....	8
<i>Діаграма класів для паттерну Bridge.....</i>	<i>13</i>
<i>Проблема, яку допоміг вирішити шаблон Bridge</i>	<i>15</i>
<i>Переваги застосування паттерну Bridge</i>	<i>16</i>
<i>Посилання на GitHub</i>	<i>17</i>
Висновок	17

Тема: Шаблони «Mediator», «Facade», «Bridge», «Template method».

Мета: ознайомитися з шаблонами проектування «Mediator», «Facade», «Bridge», «Template method» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

Варіант – 4: Графічний редактор (proxy, prototype, decorator, bridge, flyweight, SOA)
Графічний редактор повинен вміти створювати / редагувати растрові (або векторні на розсуд студента) зображення в 2-3 основних популярних форматах (bmp, png, jpg), мати панель інструментів для створення графічних примітивів, вибору кольорів, нанесення тексту, додавання найпростіших візуальних ефектів (ч/б растр, інфрачервоний растр, 2-3 на вибір учня), роботи з шарами.

Короткі теоретичні відомості

Принципи проектування:

Принцип Don't Repeat Yourself (DRY) свідчить, що в початкових кодах програмного продукту не повинно бути повторень. Ці повторення можуть спостерігатися на різних рівнях - від повторень наборів рядків коду (абсолютно аналогічні рядки в різних місцях проекту) до різної реалізації функціональності, що повторюється. Повторень уникати слід з кількох причин: передусім, код без повторень значно менше і значно легше читається; по-друге, при знаходженні помилки в коді, що має повторення, є високий шанс не знати/забути виправити цю ж помилку у іншому місці; по-третє, при створенні ідентичного коду ви не лише копіюєте функціональність, але і помилки; по-четверте, при необхідності додавання нових можливостей скрізь де є повторення необхідно внести відповідні коректури. Повторення легко виправити застосувавши відповідну техніку рефакторинга - винесення метода/інтерфейса/класа та ін.

Принцип Keep it simple, Stupid! (KISS) пропонує уникати зайвого ускладнення компонентів системи. Вважається, що система, яка складається з безлічі маленьких простих частин, працює значно надійніше, ніж одна велика складна система. Даною філософією слідує співтовариство Unix, де кожна програма виконує рівно одну функцію, але при цьому виконує її абсолютно добре (Do one thing right). Це також сприяє задоволенню відомого афоризму «The system obviously has no errors rather than system has no obvious errors» (очевидно, що в системі немає помилок; в системі немає очевидних помилок).

Використання простих конструкцій спрощує читаємість і сприйняття вихідних кодів.

Принцип You only load it once! (YOLO) вказує на необхідність підвантаження ініціалізаційних і конфігураційних змінних один раз при запуску програми, щоб уникнути проблем зі швидкістю (повторні зчитування даних з в'ячестера).

Принцип Парето застосовується до безлічі різних подій і визначає відсоткове співвідношення 80-20. Декілька прикладів застосування принципу Парето:

- 80% навантаження на сервер створює 20% додатка;
- 80% всього вихідного коду програми пишеться за 20% часу;
- 80% помилок програмного забезпечення можна усунути закривши лише 20% багів.

Таким чином, необхідно пам'ятати про те, що для вирішення більшості проблем необхідно докласти лише малу частину зусиль. Однак для доведення додатку до стану досконалості може коштувати величезну суму (80%).

Принцип You ain't gonna need it пропонує відмовитися від деяких рішень в силу того, що вони просто не знадобляться. Досить відомий факт, що програмісти люблять узагальнювати та надавати універсальні рішення на всі випадки життя. Дуже часто досить простого і працюючого рішення замість універсального, оскільки інші випадки навряд чи будуть розглядатися в подальшому. Є необхідний набір функціональності, який необхідно реалізувати; додавання зайвої гнучкості не тільки захарастить код, але і зменшить хід процесу розробки.

Шаблон «Mediator»

Структура:

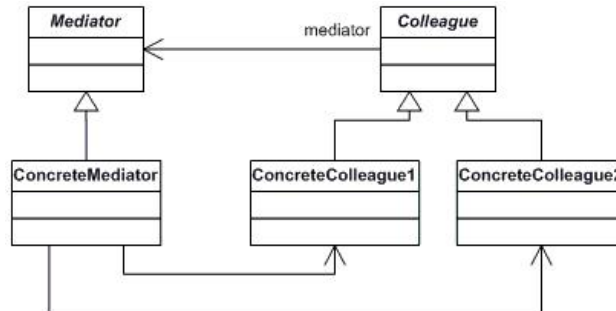


Рис.1 Структура шаблону Mediator

Призначення патерну:

Шаблон «Mediator» (посередник) використовується для визначення взаємодії об'єктів за допомогою іншого об'єкта (замість зберігання посилань один на одного). Даний шаблон схожий на шаблон «команда», проте в даному випадку замість зберігання даних про конкретну дію, зберігаються дані про взаємодії між компонентами.

Даний шаблон зручно застосовувати у випадках, коли безліч об'єктів взаємодіє між собою деяким структурованим чином, однак складним для розуміння. У такому випадку вся логіка взаємодії виноситься в окремий об'єкт. Кожен із взаємодіючих об'єктів зберігає посилання на об'єкт «медіатор».

«Медіатор» нагадує диригента при управлінні оркестром. Диригент стежить за тим, щоб кожен інструмент грав в правильний час і в злагоді з іншими інструментами. Функції «медіатора» повністю це повторюють.

Переваги та недоліки:

- + Усуває залежності між компонентами, дозволяючи повторно їх використовувати.
- + Спрощує взаємодію між компонентами.
- + Централізує управління в одному місці.
- Посередник може сильно роздуватися.

Шаблон «Facade»

Структура:

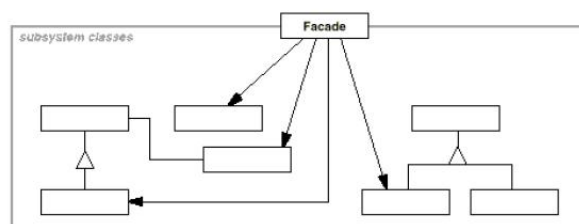


Рис.2 Структура шаблону Facade

Призначення патерну:

Шаблон «Facade» (фасад) передбачає створення єдиного уніфікованого способу доступу до підсистеми без розкриття внутрішніх деталей підсистеми. Оскільки підсистема може складатися з безлічі класів, а кількість її функцій - не більше десяти, то щоб уникнути створення «спагеті-коду» (коли все тісно пов'язано між собою) виділяють один загальний інтерфейс доступу, здатний правильним чином звертатися до внутрішніх деталей.

Це також відволікає користувачів від змін в підсистемі (внутрішня реалізація може змінюватися, а наданої послуги немає), що також скоротить кількість змін в використовуваних фасад класах (без фасаду довелося б змінювати вихідні коди в безлічі точок). Звичайно, твердої умови повного закриття внутрішніх класів підсистеми не стоїть - при необхідності можна звертатися до окремих класів безпосередньо, минаючи об'єкт фасад.

Переваги та недоліки:

- + Ізолює клієнтів від компонентів складної підсистеми.
- Фасад ризикує стати божественним об'єктом, прив'язаним до всіх класів програми.

Шаблон «Bridge»

Структура:

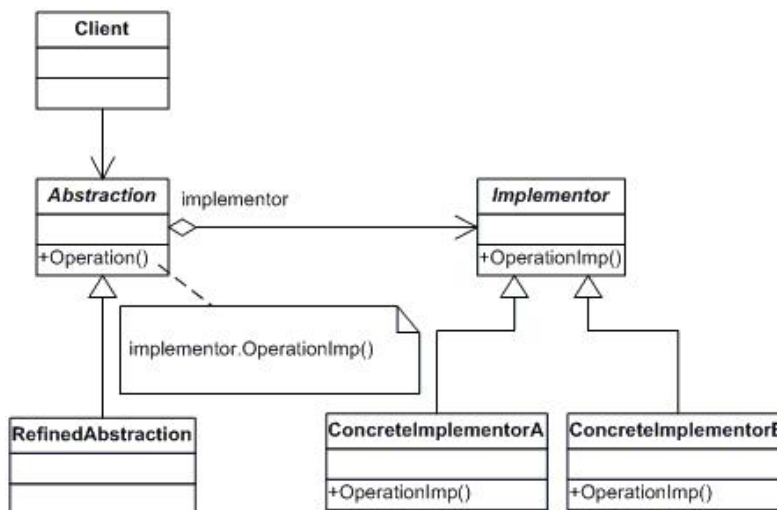


Рис.3 Структура шаблону Bridge

Призначення патерну:

Шаблон «Bridge» (міст) використовується для поділу інтерфейсу і його реалізації. Це необхідно у випадках, коли може існувати кілька різних абстракцій, над якими можна проводити дії різними способами.

В даному випадку при появі нових типів вікон буде необхідність створення нових класів їх реалізацій. Це веде до стрімкого збільшення кількості класів і плутанини в вихідному коді.

Таким чином можна відокремити абстракцію від її реалізацій, і додавати нові абстракції незалежно від реалізацій.

Переваги та недоліки:

- + Дозволяє будувати платформи-незалежні програми.
- + Приховує зайві або небезпечні деталі реалізації від клієнтського коду.
- + Реалізує принцип відкритості / закритості.
- Ускладнює код програми через введення додаткових класів.

Шаблон «Template Method»

Структура:

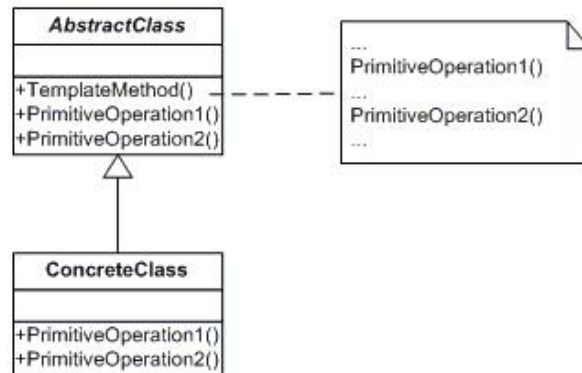


Рис.4 Структура шаблону Template Method

Призначення патерну:

Шаблон «Template Method» (шаблонний метод) дозволяє реалізувати покроково алгоритм в абстрактному класі, але залишити специфіку реалізації підкласам. Можна привести в приклад формування веб-сторінки: необхідно додати заголовки, вміст сторінки, файли, що додаються, і нижню частину сторінки. Код для додавання вмісту сторінки може бути абстрактним і реалізовуватися в різних класах - `AspNetCompiler`, `HtmlCompiler`, `PhpCompiler` і т.п. Додавання всіх інших елементів виконується за допомогою вихідного абстрактного класу з алгоритмом.

Даний шаблон дещо нагадує шаблон «фабричний метод», однак область його використання абсолютно інша - для покрокового визначення конкретного алгоритму; більш того, даний шаблон не обов'язково створює нові об'єкти - лише визначає послідовність дій.

Переваги та недоліки:

- + Полегшує повторне використання коду.
- Ви жорстко обмежені скелетом існуючого алгоритму.
- Ви можете порушити принцип підстановки Барбари Лісков, змінюючи базову поведінку одного з кроків алгоритму через підклас.
- З ростом кількості кроків шаблонний метод стає занадто складно підтримувати.

Реалізація не менше 3-х класів відповідно до обраної теми
Структура проекту з реалізованими класами зображена на рисунку 5:

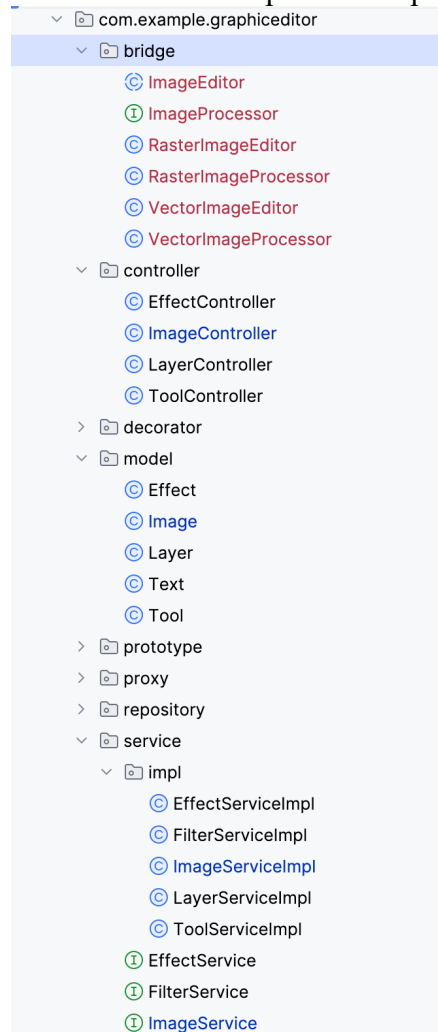


Рис.5 Структура проекту з реалізованими класами

У ході реалізації системи графічного редактора з використанням патерну bridge, було створено механізм, який дозволяє відокремити абстракцію роботи з зображеннями від конкретної реалізації алгоритмів обробки зображень. Це дозволяє легко додавати нові типи зображень та нові алгоритми обробки без необхідності змінювати основний код.

Опис реалізації:

1. Базовий клас *ImageEditor*:

- Це абстракція, яка визначає загальні операції для роботи з зображеннями, такі як застосування ефектів, зміна розміру та збереження.
- Має посилання на об'єкт *ImageProcessor*, що реалізує конкретну логіку для кожного типу зображення (растрового чи векторного).

2. Інтерфейс *ImageProcessor*:

- Описує методи для обробки зображень, які будуть конкретно реалізовані в класах *RasterImageProcessor* і *VectorImageProcessor*.
- Ці методи включають застосування ефектів, зміни розміру та збереження зображень.

3. *Конкретні реалізації ImageEditor:*

- RasterImageEditor та VectorImageEditor — це конкретні реалізації, що використовують специфічні алгоритми обробки для растрових та векторних зображень відповідно. Вони делегують виконання операцій до відповідних об'єктів ImageProcessor.

4. *Конкретні реалізації ImageProcessor:*

- RasterImageProcessor реалізує методи для обробки растрових зображень, таких як застосування ефектів, зміна розміру та збереження в таких форматах, як BMP, JPG, PNG.
- VectorImageProcessor відповідає за обробку векторних зображень, забезпечуючи їх збереження у форматі SVG.

5. *Абстракція і конкретні реалізації:*

- Використовуючи патерн bridge, реалізовано відокремлення абстракції роботи з зображеннями (через ImageEditor) від конкретної логіки обробки (через ImageProcessor). Це дозволяє розширювати систему, додаючи нові типи зображень або нові алгоритми обробки без змін у існуючій архітектурі.

6. *Гнучкість і масштабованість:*

- Завдяки такому підходу, нові типи зображень та нові алгоритми обробки можуть бути додані шляхом створення нових реалізацій класів ImageEditor та ImageProcessor, зберігаючи існуючий код незмінним.
- Це дозволяє системі бути більш гнучкою та зручною для подальших розширень, що є основною перевагою патерну bridge.

Реалізація патерну Bridge у графічному редакторі дозволила ефективно розділити абстракцію обробки зображень і конкретні реалізації для різних типів зображень та їх обробки. Це забезпечує легке розширення функціональності системи без зміни основного коду.

Реалізація шаблону за обраною темою

У ході роботи було реалізовано систему графічного редактора, в якій застосовано патерн bridge для забезпечення гнучкої архітектури, що дозволяє розширювати функціональність роботи з різними типами зображень без зміни основного коду.

Основна мета використання цього патерна — відокремити абстракцію (типи зображень) від реалізації алгоритмів обробки (методів для роботи з растровими та векторними зображеннями), що дозволяє зручно додавати нові типи зображень та алгоритми обробки без необхідності змінювати основні класи.

Опис реалізації:

1. *Клас ImageEditor (абстракція)*

Клас ImageEditor визначає основні операції для роботи з зображеннями: застосування ефектів, зміна розміру, збереження. Це абстракція, яка працює з конкретними реалізаціями через посилання на об'єкт типу ImageProcessor. Цей клас забезпечує гнучкість, дозволяючи працювати з різними типами зображень, не прив'язуючись до конкретних реалізацій.

Основні функції:

- Містить абстрактні методи applyEffect(), resize(), save(), які повинні бути реалізовані в конкретних класах.

- Містить посилання на об'єкт ImageProcessor, що делегує виконання конкретних операцій.

```

1 package com.example.graphiceditor.bridge;
2
3 public abstract class ImageEditor { 6 usages 2 inheritors
4     protected ImageProcessor imageProcessor; 7 usages
5
6     public ImageEditor(ImageProcessor imageProcessor) { 2 usages
7         this.imageProcessor = imageProcessor;
8     }
9
10    public ImageEditor() { 2 usages
11    }
12
13    public abstract void applyEffect(String effectName); 2 implementations
14
15    public abstract void resize(int width, int height); 1 usage 2 implementations
16
17    public abstract void save(String filePath); 2 implementations
18
19    public ImageEditor getEditor(String imageType) { no usages
20        return null;
21    }
22 }

```

Рис.6 Реалізація класу ImageEditor

2. Інтерфейс ImageProcessor

Інтерфейс ImageProcessor визначає набір методів для роботи з різними типами зображень. Конкретні реалізації цього інтерфейсу визначають, як будуть виконуватись операції для растрових та векторних зображень.

Основні функції:

- Методи для застосування ефектів, зміни розміру та збереження зображення.

```

1 package com.example.graphiceditor.bridge;
2
3 public interface ImageProcessor { 6 usages 2 implementations
4     void applyEffect(String effectName); 2 implementations
5     void resize(int width, int height); 2 usages 2 implementations
6     void save(String filePath); 2 implementations
7 }

```

Рис.7 Реалізація інтерфейсу ImageProcessor

3. Конкретні реалізації ImageEditor

Конкретні класи RasterImageEditor та VectorImageEditor реалізують абстракцію ImageEditor і використовують відповідні реалізації ImageProcessor для роботи з растровими та векторними зображеннями.

Основні функції:

- Делегують операції на відповідні реалізації ImageProcessor.

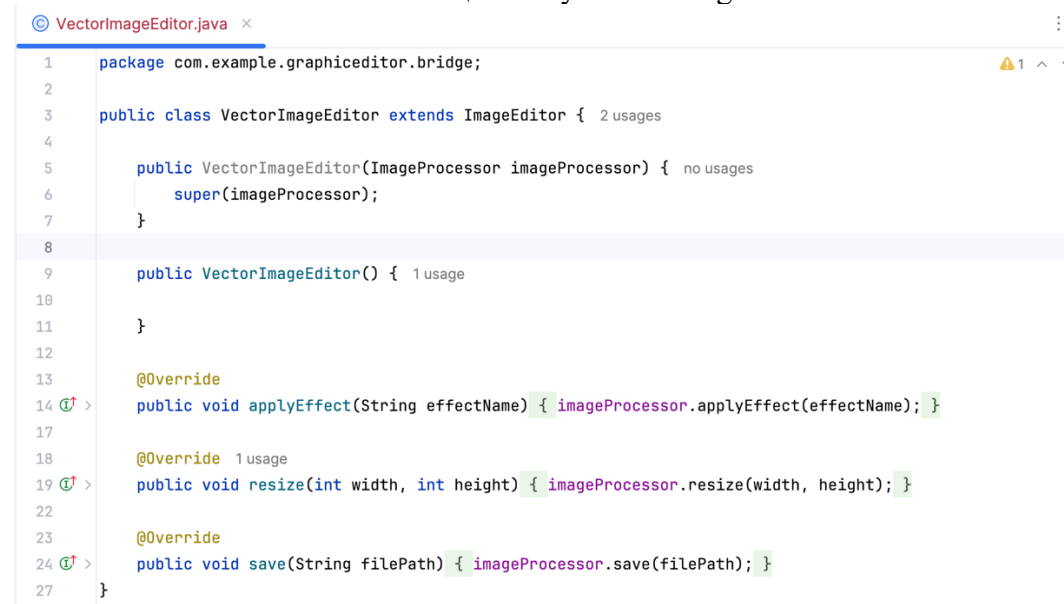


```

1 package com.example.graphiceditor.bridge;
2
3 public class RasterImageEditor extends ImageEditor { 2 usages
4
5     public RasterImageEditor(ImageProcessor imageProcessor) { no usages
6         super(imageProcessor);
7     }
8
9     public RasterImageEditor() { 1 usage
10        super();
11    }
12
13    @Override
14    public void applyEffect(String effectName) { imageProcessor.applyEffect(effectName); }
15
16
17
18    @Override 1 usage
19    public void resize(int width, int height) { imageProcessor.resize(width, height); }
20
21
22
23    @Override
24    public void save(String filePath) { imageProcessor.save(filePath); }
25
26
27 }

```

Рис.8 Реалізація класу RasterImageEditor



```

1 package com.example.graphiceditor.bridge;
2
3 public class VectorImageEditor extends ImageEditor { 2 usages
4
5     public VectorImageEditor(ImageProcessor imageProcessor) { no usages
6         super(imageProcessor);
7     }
8
9     public VectorImageEditor() { 1 usage
10
11    }
12
13    @Override
14    public void applyEffect(String effectName) { imageProcessor.applyEffect(effectName); }
15
16
17
18    @Override 1 usage
19    public void resize(int width, int height) { imageProcessor.resize(width, height); }
20
21
22
23    @Override
24    public void save(String filePath) { imageProcessor.save(filePath); }
25
26
27 }

```

Рис.9 Реалізація класу VectorImageEditor

4. Конкретні реалізації ImageProcessor

Реалізації RasterImageProcessor та VectorImageProcessor визначають, як саме виконуються операції для растрових та векторних зображень. Для растрових зображень це можуть бути формати BMP, JPG, PNG, а для векторних — формат SVG.

Основні функції:

- Реалізація методів для обробки зображень: застосування ефектів, зміна розміру, збереження.

```

1 package com.example.graphiceditor.bridge;
2
3 public class RasterImageProcessor implements ImageProcessor { no usages
4
5     @Override
6     public void applyEffect(String effectName) {
7         System.out.println("Applying raster effect: " + effectName);
8     }
9
10    @Override 2 usages
11    public void resize(int width, int height) {
12        System.out.println("Resizing raster image to " + width + "x" + height);
13    }
14
15    @Override
16    public void save(String filePath) {
17        if (filePath.endsWith(".bmp") || filePath.endsWith(".jpg") || filePath.endsWith(".png")) {
18            System.out.println("Saving raster image to: " + filePath);
19        } else {
20            throw new IllegalArgumentException("Unsupported raster file format: " + filePath);
21        }
22    }
23 }

```

Рис.10 Реалізація класу RasterImageProcessor

```

1 package com.example.graphiceditor.bridge;
2
3 public class VectorImageProcessor implements ImageProcessor { no usages
4
5     @Override
6     public void applyEffect(String effectName) {
7         System.out.println("Applying vector effect: " + effectName);
8     }
9
10    @Override 2 usages
11    public void resize(int width, int height) {
12        System.out.println("Resizing vector image to " + width + "x" + height);
13    }
14
15    @Override
16    public void save(String filePath) {
17        if (filePath.endsWith(".svg")) {
18            System.out.println("Saving vector image to: " + filePath);
19        } else {
20            throw new IllegalArgumentException("Unsupported vector file format: " + filePath);
21        }
22    }
23 }

```

Рис.11 Реалізація класу VectorImageProcessor

Завдяки застосуванню патерну bridge було досягнуто розділення абстракції та реалізації, що дозволило незалежно змінювати типи зображень і реалізації алгоритмів без необхідності змінювати існуючий код класів. Це забезпечило масштабованість і зручність в подальшому розширенні системи.

Основні переваги реалізації:

1. Гнучкість: Легко додавати нові типи зображень та алгоритми обробки, не змінюючи основний код.
2. Масштабованість: Можливість додавати нові класи для нових типів зображень чи обробки, не змінюючи існуючі класи.

3. Принципи SOLID: Реалізація патерну Bridge сприяє дотриманню принципів SOLID, зокрема принципу відкритості/закритості (Open/Closed Principle), що дозволяє розширювати систему без зміни існуючого коду.
4. Модульність: Оскільки кожен клас відповідає за конкретну частину функціональності, система є модульною та зручною для підтримки.

Використання патерну bridge дозволило створити гнучку та масштабовану архітектуру для роботи з різними типами зображень у графічному редакторі. Реалізація цього патерну забезпечила можливість додавання нових типів зображень і алгоритмів без зміни основних класів, що значно спрощує розширення та підтримку системи.

Діаграма класів для паттерну Bridge

Діаграма класів, яка реалізовує патерн bridge зображена на рисунку 12:

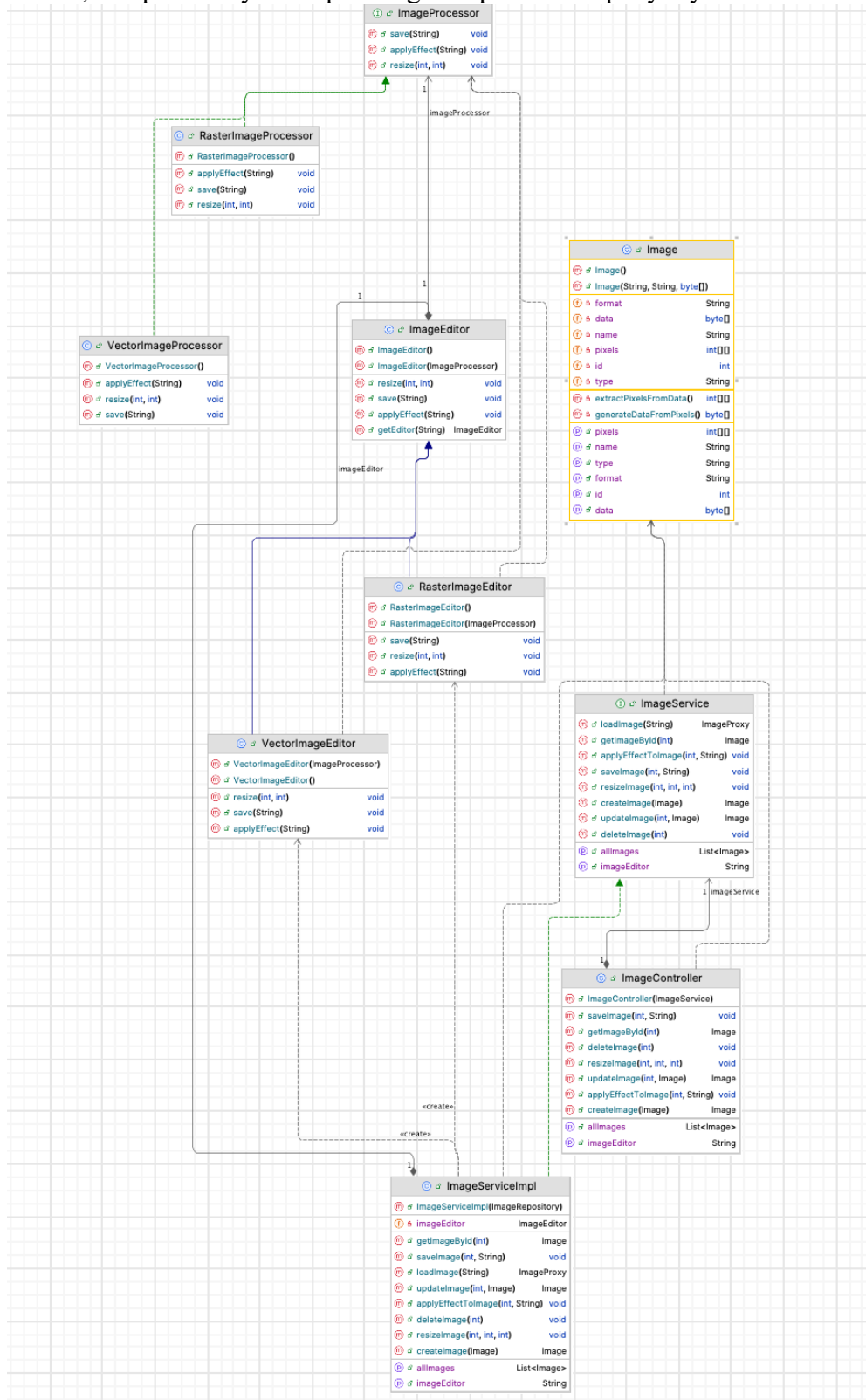


Рис.12 Діаграма класів для паттерну Bridge

Ця діаграма класів представляє реалізацію патерну Bridge для роботи з растровими та векторними зображеннями у графічному редакторі.

Основні компоненти:

Абстракція:

1. ImageEditor (Абстрактний клас)

Виступає в ролі високорівневої абстракції для редагування зображень.

Методи:

- `resize(int, int)` — змінює розмір зображення.
- `applyEffect(String)` — застосовує ефект до зображення.
- `save(String)` — зберігає зображення.
- `getEditor(String)` — визначає конкретний редактор залежно від типу зображення.

2. RasterImageEditor і VectorImageEditor (Класи)

- Конкретні реалізації ImageEditor, які працюють з растровими та векторними зображеннями відповідно.
- Визначають специфічну логіку для обробки зображень залежно від їхнього типу.

Реалізація:

3. ImageProcessor

Визначає інтерфейс для низькорівневої обробки зображень.

Методи:

- `save(String)` — зберігає файл.
- `applyEffect(String)` — додає ефект.
- `resize(int, int)` — змінює розмір зображення.

4. RasterImageProcessor і VectorImageProcessor (Класи)

Конкретні реалізації ImageProcessor для роботи з растровими та векторними зображеннями. Відповідають за виконання низькорівневих операцій, таких як зміна розміру або застосування ефектів.

Зв'язки:

Клас ImageEditor залежить від об'єкта типу ImageProcessor. Це дозволяє незалежно розширювати як абстракцію (ImageEditor), так і реалізацію (ImageProcessor).

Наприклад:

- RasterImageEditor використовує RasterImageProcessor для роботи з растровими зображеннями.
- VectorImageEditor взаємодіє з VectorImageProcessor для обробки векторних зображень.

Допоміжні компоненти:

5. Image (Клас)

Визначає структуру даних для зображень, включаючи формат, пікселі, тип і ідентифікатор. Методи для конвертації між пікселями та даними (`extractPixelsFromData`, `generateDataFromPixels`).

6. ImageService (Інтерфейс)

Визначає бізнес-логіку для маніпуляцій із зображеннями, використовуючи редактори (ImageEditor).

7. ImageServiceImpl (Клас)

Реалізує ImageService та координує роботу між редакторами та об'єктами зображень.

8. ImageController (Клас)

Контролер, який забезпечує взаємодію користувача із сервісом через API.

Методи:

- saveImage
- applyEffectToImage
- resizeImage
- deleteImage, updateImage

Патерн Bridge дозволяє легко розширювати як редактори зображень (ImageEditor), так і процесори (ImageProcessor), забезпечуючи гнучкість і зменшення залежностей між високорівневою абстракцією та низькорівневою реалізацією.

Проблема, яку допоміг вирішити шаблон Bridge

Шаблон bridge у цьому графічному редакторі допоміг вирішити кілька важливих проблем, пов'язаних із організацією роботи з різними типами зображень.

1. Розділення абстракції та реалізації

Шаблон Bridge забезпечив чітке розділення між:

- Абстракцією (ImageEditor), яка визначає високорівневі операції, такі як applyEffect(), resize() і save().
- Реалізацією (ImageProcessor), яка відповідає за конкретну логіку обробки зображень (растрових або векторних).

Це дозволило незалежно змінювати або розширювати функціональність як у редакторах (RasterImageEditor, VectorImageEditor), так і в процесорах (RasterImageProcessor, VectorImageProcessor).

2. Гнучкість у роботі з різними типами зображень

Bridge дозволив підтримувати різні типи зображень (растрові, векторні) без дублювання коду:

- Логіка для растрових зображень реалізована у RasterImageProcessor.
 - Логіка для векторних зображень реалізована у VectorImageProcessor.
- При цьому обидва типи можуть використовувати спільний інтерфейс ImageProcessor і однакові методи редакторів.

3. Уникнення великої кількості класів

Без використання шаблону Bridge необхідно було б створювати окремі класи для кожного поєднання типів зображень і їхньої обробки, наприклад:

- RasterImageEditorWithBasicProcessor.
- VectorImageEditorWithAdvancedProcessor.

Це б призвело до складної ієрархії класів. Завдяки Bridge достатньо створювати окремі класи для редакторів та обробників і поєднувати їх динамічно.

4. Легка підтримка та масштабування

- Додавання нового типу зображення (наприклад, 3D-зображення) або нового способу обробки не потребує змін у вже існуючих класах.
- Можна додати новий обробник (наприклад, ThreeDImageProcessor) або новий редактор (ThreeDImageEditor) без впливу на інші компоненти.

5. Спрощене тестування

Оскільки абстракція (ImageEditor) і реалізація (ImageProcessor) розділені, їх можна тестувати окремо:

- Логіку роботи редактора (наприклад, делегування викликів методам обробника) можна тестувати незалежно.
- Логіку обробки (наприклад, застосування ефектів або збереження зображень) можна перевіряти в конкретних реалізаціях ImageProcessor.

Шаблон Bridge оптимізував роботу з графічним редактором, забезпечивши:

1. Розширюваність системи без внесення змін у вже існуючі класи.
2. Уніфікацію способу роботи з різними типами зображень.
3. Уникнення дублювання коду та зайвої складності ієрархії.
4. Гнучкість у додаванні нових типів зображень або методів обробки.

Завдяки шаблону Bridge система стала більш гнучкою, масштабованою та легкою в підтримці.

Переваги застосування патерну Bridge

Переваги використання патерну Bridge у графічному редакторі:

1. Поділ абстракції та реалізації

- Завдяки Bridge вдалося розділити логіку роботи редактора (абстракцію) та механізми обробки зображень (реалізацію).
- Це дозволяє незалежно вдосконалювати редактори або процесори зображень, не впливаючи на інші компоненти системи.

2. Спрощення структури проєкту

- Патерн допоміг уникнути створення великої кількості класів для кожного поєднання типів зображень і методів їх обробки.
- Наприклад, замість дублювання логіки збереження та редагування у кожному типі редактора, ці операції делегуються відповідним процесорам.

3. Масштабованість

- Легко додавати нові типи зображень (наприклад, 3D-зображення) або способи їх обробки, створюючи окремі класи, що відповідають лише за свою частину роботи.
- Наприклад, новий формат зображень можна реалізувати шляхом створення нового класу ImageProcessor, не змінюючи код редакторів.

4. Гнучкість у налаштуванні

- Абстракція та реалізація можуть поєднуватися у різних комбінаціях. Один редактор може працювати з різними процесорами, залежно від потреб, що забезпечує високий рівень адаптивності системи.

5. Полегшення модифікації

- Зміни у функціоналі процесора або редактора впливають лише на відповідний клас, зменшуючи ризик помилок у програмі.
- Наприклад, якщо потрібно змінити логіку масштабування растрових зображень, це можна зробити у класі RasterImageProcessor, не зачіпаючи інші компоненти.

6. Поліпшення тестування

- Компоненти Bridge (процесори та редактори) є незалежними, що дозволяє тестувати їх окремо.
- Наприклад, можна перевірити збереження файлів у класі VectorImageProcessor, не тестуючи при цьому редактори.

7. Простота інтеграції нових функцій

- Додавання нових можливостей, таких як обробка спеціальних ефектів чи підтримка нових форматів файлів, потребує створення лише нового класу процесора або редактора, без змін у вже існуючих класах.

8. Зниження складності

- Використання композиції замість наслідування дозволило створити більш прозору та зрозумілу архітектуру, де кожен компонент має чітко визначену роль.
- Це полегшує роботу над проєктом як для розробників, так і для нових членів команди.

Завдяки патерну Bridge графічний редактор став більш гнучким та ефективним. Він забезпечує просте управління складністю системи, дозволяє легко розширювати функціонал і підтримувати код, не створюючи надмірної залежності між компонентами.

Посилання на GitHub

Посилання на репозиторій: <https://github.com/ElLviraaaaaa/GraphicEditor.git>

Висновок

У даній роботі я реалізувала патерн проєктування Bridge для графічного редактора, що дозволило розділити абстракцію (редактори зображень) та реалізацію (процесори обробки зображень). Завдяки цьому шаблону вдалося створити гнучку систему, яка підтримує різні типи зображень (растрові, векторні) та способи їх обробки без необхідності змінювати існуючий код.

Застосування Bridge допомогло знизити складність системи, уникаючи великої кількості класів, які б поєднували різні варіанти редакторів і процесорів. Це зробило програму більш масштабованою, дозволивши легко додавати нові типи зображень або функціонал, а також забезпечило модульність і спрощену підтримку. Завдяки Bridge графічний редактор став ефективним, адаптивним та простим у розширенні.